MEMORIA TÉCNICA | 2026

Login, dashboard y conexión con Supabase

Aplicación Ionic y Angular

Autor: Mateo Fernández Blasco

Librerías: Ionic Angular y Supabase JS

Fecha: 27 de octubre de 2026

Alcance: autenticación, métricas y conexión de datos

Alcance de esta memoria

Este documento se centra exclusivamente en el formulario de login, el dashboard y la conexión de la aplicación con Supabase. Describe las librerías utilizadas, la configuración del cliente, las consultas de datos y el flujo entre la interfaz y la base de datos. No incluye otras pantallas del proyecto.

Librerías utilizadas

Librería	Uso en login/dashboard/Supabase
@ionic/angular	Componentes de interfaz: campos de entrada, botones, tarjetas, cabeceras, indicadores de carga y notificaciones Toast.
@angular/forms	Formularios reactivos del login y validadores de correo obligatorio, formato de email y contraseña obligatoria.
@angular/router	Navegación después del login y protección de rutas con una guardia.
@supabase/supabase-js	Cliente JavaScript que conecta Angular con Supabase y permite consultar la tabla empleados.
Angular signals / DI	Inyección de servicios y representación de datos/estado en los componentes donde aplica.

1. Login: validación y flujo

La página de acceso se implementa en `login.page.ts` y su plantilla. Usa un formulario reactivo con campos de correo y contraseña. Antes de consultar Supabase, el formulario valida que los datos tengan el formato requerido. Mientras se procesa la petición, `isSubmitting` evita envíos repetidos y permite mostrar un spinner.

Secuencia de acceso

- El usuario introduce correo y contraseña.
- El componente LoginPage llama a `AuthService.login(email, password)`.
- El servicio normaliza el correo (espacios exteriores y minúsculas) y busca el registro en `empleados`.
- Si encuentra coincidencia, construye un objeto Usuario con id, nombre, correo y rol y lo guarda en `localStorage` con la clave `usuario_sesion`.
- La página navega a `/dashboard` para Admin o `/usuarios` para User. Si no hay coincidencia, muestra un aviso de credenciales incorrectas; si falla la petición, muestra un mensaje de conexión.

Consulta implementada

La comprobación actual se hace mediante una consulta a la tabla `empleados`: selección de campos de perfil, filtro insensible a mayúsculas del email (`ilike`) y comparación con la columna `contraseña` (`eq`), terminando con `maybeSingle()`. El proyecto actual no utiliza `supabase.auth.signInWithPassword()` para este login; consulta directamente la tabla de datos.

2. Dashboard: lectura y métricas

Al iniciarse, el dashboard recupera la sesión almacenada localmente, obtiene el rol mediante AuthService y llama a `getEmpleados()`. Mientras espera, presenta un indicador de carga. Al recibir los datos calcula los valores que se muestran en sus tarjetas.

Indicadores

- Plantilla total: número de registros devueltos por Supabase.
- Presupuesto salarial: suma de los salarios, convirtiendo los valores a número y evitando que valores no numéricos alteren la suma.
- Empleados activos: cuenta los registros cuyo campo `activo` es verdadero.
- La vista usa CurrencyPipe para mostrar la suma como moneda EUR.

Consulta del dashboard

`AuthService.getEmpleados()` consulta `empleados`, solicita los campos `id, created_at, nombre, email, edad, salario, activo, rol` y ordena ascendentemente por id. Si Supabase devuelve un error, el servicio lo registra y lo propaga; el dashboard lo captura, registra y apaga el indicador de carga en un bloque `finally`.

Sesión y navegación

El método `logout()` limpia el estado del usuario y elimina `usuario_sesion` de localStorage; el dashboard redirige al login. La guardia de rutas consulta `isAuthenticated()`, que en la implementación actual depende del objeto cargado/establecido por el servicio.

3. Conexión completa con Supabase

La integración se centraliza en dos archivos. El cliente se crea una sola vez y el servicio AuthService reutiliza esa instancia para el login y las consultas del dashboard.

Configuración y cliente

Archivo	Responsabilidad
`src/app/environments/environment.ts`	Define los valores de configuración `supabaseUrl` y `supabaseKey`. Los valores concretos se omiten en esta memoria.
`src/app/services/supabase.client.ts`	Importa `createClient` desde `@supabase/supabase-js` e inicializa y exporta el cliente con la URL y la clave del entorno.
`src/app/services/auth.ts`	Centraliza las operaciones de login y acceso a empleados, tratamiento de errores y sesión local.

Cadena de conexión

- El archivo de entorno proporciona URL del proyecto Supabase y clave pública de cliente.
- `supabase.client.ts` llama a `createClient(environment.supabaseUrl, environment.supabaseKey)`.
- `AuthService` importa la instancia supabase desde el cliente compartido.
- Las llamadas `supabase.from("empleados")` se convierten en solicitudes al API de Supabase; las respuestas contienen datos y, cuando procede, un objeto de error.

Esquema esperado por las pantallas

La tabla `empleados` debe tener, como mínimo, los campos consultados por el login y dashboard: `id`, `nombre`, `email`, `contraseña`, `rol`, `created_at`, `edad`, `salario` y `activo`. La columna de contraseña es necesaria para la consulta de login tal como está programada.

Manejo de errores

El servicio registra y propaga los errores de lectura de empleados; el login distingue la ausencia de coincidencia de una excepción de conexión. El dashboard apaga el estado de carga incluso si la consulta falla. Las políticas RLS del proyecto deben permitir, limitar o denegar cada consulta según corresponda; una clave pública no sustituye la autorización de base de datos.

4. Ejecución, validación y seguridad

Ejecución

Desde la carpeta del proyecto `gestion-multiplataforma`, los comandos habituales son:

```
npm install
```

```
npm start
```

```
npm run build
```

Validación registrada

En la sesión del 27 de octubre de 2026 se ejecutó `npm run build` para gestion-multiplataforma y terminó correctamente. El build mostró un aviso de Browserslist por versiones antiguas de navegadores; no bloqueó la generación del bundle. Esta comprobación valida compilación, no las credenciales reales, la conectividad del proyecto Supabase ni sus políticas RLS.

Consideraciones de seguridad importantes

- Contraseña: el login actual compara directamente la columna `contraseña` desde el cliente web. Si esa columna almacena texto sin hash, las credenciales quedan expuestas a usuarios con acceso a la consulta o a la tabla. No es adecuado para producción.
- Recomendación: autenticar con Supabase Auth (`signInWithPassword`) y asociar el usuario autenticado a un perfil de empleado. No consultar ni devolver contraseñas desde el navegador.
- Clave: la clave pública usada en una aplicación cliente no es un secreto; no debe confundirse con una service-role key. La autorización se debe aplicar mediante RLS y políticas restrictivas.
- Sesión local: almacenar un objeto de usuario en localStorage sirve para conservar estado de interfaz, pero no es por sí mismo una prueba de autenticación segura. La guardia frontend no sustituye la autorización en Supabase.
- Configuración: no publicar contraseñas ni claves privilegiadas. Los datos de configuración del cliente pueden quedar accesibles en el bundle de frontend.

Conclusión

El login y el dashboard utilizan una instancia compartida de Supabase JS, configurada en el entorno y creada mediante `createClient`. El servicio realiza la consulta de credenciales y obtiene empleados; el login decide la navegación por rol y el dashboard calcula sus indicadores a partir de esos registros. La compilación está verificada. Antes de un uso real, conviene migrar la autenticación a Supabase Auth y definir políticas RLS seguras para la tabla de empleados.